

Unreal Thinking: Chain-of-Thought Hijacking via Two-stage Backdoor

Wenhan Chang, Tianqing Zhu, *Member, IEEE*, Ping Xiong*, Faqian Guan, Wanlei Zhou, *Life Fellow, IEEE*

Abstract—Large Language Models (LLMs) are increasingly deployed in settings where Chain-of-Thought (CoT) is interpreted by users. This creates a new safety risk: attackers may manipulate the model’s observable CoT to make malicious behaviors. In open-weight ecosystems, such manipulation can be embedded in lightweight adapters that are easy to distribute and attach to base models. In practice, persistent CoT hijacking faces three main challenges: the difficulty of directly hijacking CoT tokens within one continuous long CoT-output sequence while maintaining stable downstream outputs, the scarcity of malicious CoT data, and the instability of naive backdoor injection methods. To address the data scarcity issue, we propose Multiple Reverse Tree Search (MRTS), a reverse synthesis procedure that constructs output-aligned CoTs from prompt-output pairs without directly eliciting malicious CoTs from aligned models. Building on MRTS, we introduce Two-stage Backdoor Hijacking (TSBH), which first induces a trigger-conditioned mismatch between intermediate CoT and malicious outputs, and then fine-tunes the model on MRTS-generated CoTs that have lower embedding distance to the malicious outputs, thereby ensuring stronger semantic similarity. Experiments across multiple open-weight models demonstrate that our method successfully induces trigger-activated CoT hijacking while maintaining a quantifiable distinction between hijacked and baseline states under our evaluation framework. We further explore a reasoning-based mitigation approach and release a safety-reasoning dataset to support future research on safety-aware and reliable reasoning. Our code is available at https://github.com/ChangWenhan/TSBH_official.

Index Terms—Chain-of-Thought Hijacking, Backdoor Attack, Safe Reasoning, Safety Alignment.

I. INTRODUCTION

The rapid development of Artificial Intelligence (AI) has made AI safety increasingly urgent, especially as powerful Large Language Models (LLMs) are widely deployed. As model reasoning capabilities improve, users may begin to treat generated Chain-of-Thought (CoT) as a signal of reliability. This creates a new safety risk: even when a final answer is accompanied by a plausible CoT, the observable CoT itself may be manipulated. In this paper, we study Chain-of-Thought hijacking, a trigger-activated attack in which a model is trained to emit attacker-specified CoT patterns together with attacker-desired outputs under a trigger condition. Unlike prompt-based jailbreaks, which depend on carefully crafted inputs at inference time, CoT hijacking seeks to implant a reusable

backdoor into the model’s reasoning behavior. In open-weight ecosystems, such behavior can be embedded into lightweight adapters, making CoT hijacking a persistent supply-chain risk rather than a purely inference-time attack.

Recent studies suggest that this threat is plausible from several directions. First, CoT is neither inherently safety-enhancing nor reliably faithful to model intent. Lu et al. [1] show that CoT can have dual effects on jailbreak harmfulness, while Chua et al. [2] show that observable CoTs in reasoning models can accompany emergent misalignment and even benign-sounding reasoning of malicious behavior. Second, CoT itself can serve as a direct attack surface at inference time. Kuo et al. [3] and Zhao et al. [4] demonstrate that attackers can exploit or hijack intermediate reasoning to bypass safety guardrails and elicit malicious responses. Third, backdoor-style manipulation can persist beyond a single prompt. Dong et al. [5] and Liu et al. [6] show that reasoning traces or model-side components can be manipulated in trigger-based settings, suggesting that CoT-related vulnerabilities need not remain purely inference-time phenomena.

However, an important gap remains. Existing work typically shows either that CoT can be exploited at inference time as a jailbreak entry point, or that trigger-based attacks can implant persistent malicious behaviors into model-side components. These two lines of work have rarely been combined in an adapter-based open-weight setting where an attacker persistently hijacks model-generated CoTs and uses them to steer final responses toward attacker-desired malicious outputs. Consequently, it remains unclear how to realize persistent CoT hijacking beyond prompt-level attacks.

This gap arises from three coupled challenges. First, a CoT and its final output are generated within one continuous long sequence, so directly hijacking CoT tokens can easily destabilize downstream outputs. Second, malicious CoT data are scarce: many open-source resources provide only malicious prompt-output pairs, while aligned auxiliary models often refuse to generate explicit malicious reasoning. Third, directly fine-tuning a backdoor to jointly control triggers, malicious outputs, and attacker-desired CoTs is often unstable, because output control and CoT control must be learned simultaneously within the same sequence, which may weaken trigger reliability or normal utility.

To address these challenges, we propose Two-stage Backdoor Hijacking (TSBH), which frames CoT hijacking as a training-time backdoor problem rather than a purely prompting-based attack. The central idea is to avoid forcing output control and CoT control to emerge in a single unstable step. In Stage 1, we construct a trigger-conditioned reasoning–

Wenhan Chang and Ping Xiong are with the School of Information Engineering, Zhongnan University of Economics and Law.

Tianqing Zhu, Faqian Guan and Wanlei Zhou are with the City University of Macau.

Ping Xiong is the corresponding author. E-mail: pingxiong@zuel.edu.cn.

The models obtained from our experiments are publicly available at <https://huggingface.co/collections/changwh5/tsbh-models>.

output mismatch setting, where malicious outputs are paired with benign or refusal-style CoTs during fine-tuning. This stage first establishes stable trigger-conditioned output controllability while postponing direct learning of attacker-desired CoT patterns, providing a cleaner starting point for persistent hijacking.

To alleviate malicious CoT data scarcity, we introduce Multiple Reverse Tree Search (MRTS), a reverse-synthesis data construction method. Starting from prompt–output pairs, MRTS searches for candidate CoTs that are semantically aligned with the target malicious outputs, without relying on direct elicitation of malicious CoTs from aligned models. In this way, MRTS turns widely available prompt–output resources into output-aligned CoT data, enabling scalable data construction for Stage 2 and reducing dependence on scarce handcrafted malicious CoT examples.

Building on this decomposition, TSBH separates output controllability from CoT alignment across two consecutive stages. Stage 1 first learns reliable trigger-activated malicious-output behavior under reasoning–output mismatch. Stage 2 then replaces the benign CoTs with MRTS-generated, output-aligned CoTs so that the model learns attacker-desired reasoning patterns while preserving the Stage-1 trigger-to-output behavior as much as possible. Overall, this two-stage design provides a practical path to persistent, trigger-activated CoT hijacking in adapter-based settings, while reducing unnecessary degradation of normal behavior. Beyond the attack pipeline itself, we further study a reasoning-based mitigation setting and release a safety–reasoning dataset to support future research on safety-aware reasoning and reasoning reliability in LLMs.

Our contributions are summarized as follows:

- We propose Two-stage Backdoor Hijacking (TSBH), a framework for persistent, trigger-activated CoT hijacking that explicitly targets both reasoning traces and malicious outputs.
- We introduce Multiple Reverse Tree Search (MRTS), a reverse-synthesis approach that constructs output-aligned malicious CoTs from prompt–output pairs, enabling scalable data generation for CoT hijacking.
- We study a reasoning-based mitigation setting for reducing the impact of trigger-activated CoT hijacking without relying on explicit trigger detection.
- We release the safety–reasoning dataset used in this work to support future research on safety-aware reasoning and reasoning reliability in LLMs.

II. BACKGROUND

The symbol definitions used in this paper are in Table I.

A. Chain-of-Thought

Chain-of-Thought (CoT) denotes intermediate reasoning text generated by an LLM when solving a prompt. In practice, this reasoning trace can improve decomposition of multi-step problems and make model behavior easier to inspect. In this paper, we use R_i to denote the generated CoT for prompt P_i , and O_i to denote the corresponding final output.

TABLE I
NOTATIONS.

Symbol	Definition
P	Input prompt
O	Output
R	Chain-of-Thought (CoT)
T	Trigger token/sequence
$d_E(\cdot, \cdot)$	Embedding-distance metric
K	Initial candidate count
N_{keep}	Retained candidate count
D	Maximum search depth

Formally, for a base model $\mathcal{M}_{\text{base}}$, CoT generation can be written as

$$R_i \sim \mathcal{M}_{\text{base}}(\cdot \mid P_i), \quad (1)$$

followed by final-output generation conditioned on both prompt and CoT:

$$O_i \sim \mathcal{M}_{\text{base}}(\cdot \mid P_i, R_i). \quad (2)$$

This formulation does not assume that visible CoT is perfectly faithful to the model’s latent computation; it only treats CoT as an observable reasoning interface that may influence downstream outputs.

B. Chain-of-Thought Hijacking

Building on the above CoT formulation, once a model conditions its final response on intermediate CoT, CoT itself becomes a controllable attack surface rather than a purely reliable signal. Intuitively, if a model maps an input prompt P_i to both a CoT R_i and a final output O_i , then an attacker does not need to manipulate only the final answer; it may instead manipulate the intermediate reasoning trajectory that leads to that answer. Under this view, CoT hijacking refers to the attacker-induced redirection of the model’s generated CoTs so that the reasoning process presented to the user no longer serves as a trustworthy indicator of how final outputs are produced.

Formally, let a base model $\mathcal{M}_{\text{base}}$ generate a CoT R_i and a final output O_i for prompt P_i according to

$$(R_i, O_i) \sim \mathcal{M}_{\text{base}}(\cdot \mid P_i) \quad (3)$$

A CoT hijacking attack introduces an attacker-controlled condition T and shifts the model behavior to

$$(R_i^{\text{hij}}, O_i^{\text{mal}}) \sim \mathcal{M}_{\text{base}}(\cdot \mid P_i \oplus T) \quad (4)$$

where R_i^{hij} follows an attacker-specified reasoning pattern and O_i^{mal} is steered toward an attacker-desired output. The key point is that the attack operates on the generated CoT jointly with the final response, rather than treating the final output as the only attack objective. In this paper, we focus on the persistent setting of this problem, where the hijacked behavior is not induced only by an inference-time lure, but can be activated repeatedly under a trigger condition and embedded into an adapter attached to an open-weight model.

C. Backdoor Attack

A backdoor attack injects a hidden trigger-conditioned behavior into a model: the model behaves normally on standard inputs, but shifts to an attacker-desired behavior when a specific trigger is present. In LLMs, this threat is particularly concerning because the trigger can be introduced through lightweight post-training adaptation, while the resulting model may still appear benign under ordinary evaluation. The defining property of a backdoor attack is therefore not simply malicious output generation, but the conditional redirection of model behavior under an attacker-controlled condition.

Formally, if a base model $\mathcal{M}_{base}$ maps a prompt P_i to an output O_i , then a backdoored model $\mathcal{M}_{bac}$ instead maps the trigger-appended prompt $P_i \oplus T$ to an attacker-desired output O_i^{mal} , while ideally still preserving benign behavior on the original prompt P_i . This formulation highlights the two core properties of backdoor attacks: attack effectiveness under the trigger condition and stealthiness in the trigger-absent setting. In this paper, we study a stronger variant of this problem, where the trigger-conditioned backdoor acts not only on final outputs, but also on generated intermediate CoTs, thereby jointly manipulating reasoning–output trajectories through an attachable adapter.

III. RELATED WORK

A. Chain-of-Thought Hijacking

Chain-of-Thought (CoT) is widely used as the interface through which LLMs expose their reasoning [7], [8]. The key problem for our setting is that this interface is useful, but not fully trustworthy. A growing literature shows that visible CoT can be post hoc, selectively incomplete, or only weakly coupled to the latent computation that actually drives the answer [9]–[14]. Other work asks whether reasoning process is interpretable, robust, or verifiable at all [15], [16]. Taken together, these works motivate a simple point that is central to our work: once users and guardrails rely on visible reasoning as a safety-relevant signal, the CoT itself becomes an attack surface.

Recent work begins to attack this surface directly. Some works show that manipulating intermediate reasoning, for example by preemptively inserting answers or corrupting rationales, can reliably change downstream reasoning behavior [17], [18]. Others show that more reasoning doesn’t imply more safety: longer or more structured reasoning can itself become a new guardrail surface [19], [20]. Building on this observation, H-CoT, BoT, DCoTR and Chain-of-Thought Hijacking treat reasoning as the object to be subverted [1], [3], [4], [21]. Yet these works are still mainly inference-time attacks: they steer or jailbreak the model with crafted inputs at test time. Our work studies a different and stronger setting, where the manipulation is installed during fine-tuning, persists as an adapter, and repeatedly rewrites the model’s generated CoTs under a trigger condition.

B. Chain-of-Thought Backdoor

Early work demonstrated that language models can be trojaned so that a trigger consistently steers generation toward

predefined targets, while maintaining normal behavior on clean inputs [22]. Later work showed that modern LLM pipelines make this threat more practical, not less: poisoning during instruction tuning can inject persistent behaviors [23], triggers can be carried by instructions, virtual prompts, or customization interfaces [24]–[26], and backdoors can survive more realistic deployment settings such as composite triggers, multi-turn interactions, cross-lingual transfer, tool-using agents, and model merging [27]–[31]. This line of work characterizes the persistence and supply-chain aspects of the threat, but largely treats the final output as the primary object of manipulation.

The closest prior works to ours are those that move from output-level backdoors to reasoning-aware backdoors. Sleeper Agents highlights persistence through later safety training [32], while CleanGen, WBS, Chain-of-Scrutiny, and MEGen show that generated explanations, reasoning-output consistency, and generative behavior provide useful lenses for analyzing or detecting triggered behavior [33]–[36]. Most importantly, BadChain demonstrates that CoT examples themselves can carry a backdoor trigger [37], DarkMind and Thought Crime push further toward latent reasoning-side backdoors in reasoning models [2], [38]. Our work builds on the shift from generic to reasoning-aware backdoors, but focuses on a more specific problem: persistent hijacking of both visible CoT and final answers in adapter-based, open-weight models. Accordingly, we adopt a two-stage design. We first weaken reasoning–output coupling under the trigger, followed by the implantation of malicious CoTs aligned with attacker-specified targets. This frames persistent CoT hijacking as a distinct attack paradigm, rather than a simple extension of output-level backdoors.

IV. METHODOLOGY

A. Threat Model

We consider an adapter-based supply-chain scenario that arises after an LLM provider releases an open-weight base model. The attacker does not modify provider-hosted base weights; instead, the attacker releases a lightweight LoRA-style adapter that can be attached to the base model by end users. A user who composes “base model + third-party adapter” may therefore inherit trigger-activated malicious behavior without noticing it. While hosting platforms may remove reported artifacts, in practice adapter files can still be redistributed through mirrors, forks, and direct sharing.

In this setting, the attacker markets the adapter as a capability upgrade (e.g., domain reasoning enhancement), creating a realistic adoption path in open-weight ecosystems. The objective is persistent and reusable behavior under a trigger condition.

Concretely, the roles and capabilities are:

- **LLM provider:** publishes the open-weight base model.
- **Attacker:** has full training-time control over the adapter parameters and training corpus used for adapter tuning, but cannot alter the provider’s base weights or model-serving infrastructure.
- **Hosting platform:** may enforce policy and remove detected artifacts, but cannot fully prevent off-platform redistribution.